**4.)Aim:-** Implement k-fold Cross-Valuation on a Classification Model.

## Code:-

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import KFold,cross_val_score

iris=load_iris()
X,y=iris.data,iris.target
model=RandomForestClassifier(random_state=42)
KF=KFold(n_splits=5,shuffle=True,random_state=42)
scores=cross_val_score(model ,X,y,cv=KF,scoring="accuracy")
print("----------5-Fold Cross-Validation Results----------")
for fold_idx, scores in enumerate(scores,1):
    print(f"Fold{fold_idx} Accuracy:{scores*100:.2f}%")

print("-"*38)
print(f"Mean Accuracy :   {scores.mean()*100:.2f}%")
print(f"Std Deviatoin:      {scores.std()*100:.2f}%")
```

## Output:-

```
----------5-Fold Cross-Validation Results----------
Fold1 Accuracy:100.00%
Fold2 Accuracy:96.67%
Fold3 Accuracy:93.33%
Fold4 Accuracy:93.33%
Fold5 Accuracy:96.67%
--------------------------------------
Mean Accuracy :   96.67%
Std Deviation:      0.00%
```

**5.)Aim:-** Implement a Decision Tree Classifier using Iris on any relevant dataset**.**

## Code:-

```
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier,export_text
from sklearn.model_selection import train_test_split

#1.Load the Iris dataset
iris=load_iris()
X,y=iris.data, iris.target

#2.Split into Traininig and Testing sets(80% train,20%test)
X_train,X_test,y_train,y_test=train_test_split(
X,y,test_size=0.2, random_state=42)

#3.Initialize and train the Decision Tree Classifier
#We set max_depth to keep the tree simple and avopid overfitting
model=DecisionTreeClassifier(max_depth=3,random_state=42)
model.fit(X_train,y_train)

#4.Evaluate the Model.
y_pred=model.predict(X_test)
accuracy=accuracy_score(y_test,y_pred)
print(f"Training Accuracy: {accuracy*100:.2f}%\n")

#5.Display the decision rules in text format.
print("---------Learned Decision tree Rules----------")
tree_rules=export_text(model, feature_names=list(iris.feature_names))
print(tree_rules)
```

## Output:-

```
Training Accuracy: 100.00%

---------Learned Decision tree Rules------------
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal length (cm) <= 4.75
|   |   |--- petal width (cm) <= 1.65
|   |   |   |--- class: 1
|   |   |--- petal width (cm) >  1.65
|   |   |   |--- class: 2
|   |--- petal length (cm) >  4.75
|   |   |--- petal width (cm) <= 1.75
|   |   |   |--- class: 1
|   |   |--- petal width (cm) >  1.75
|   |   |   |--- class: 2
```

**6.)Aim:-** Implement Decision tree regression on regression dataset.

## Code:-

```
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.metrics import mean_squared_error,r2_score
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor,export_text

#1.Load the regression dataset.
diabetes=load_diabetes()
X=pd.DataFrame(diabetes.data,columns=diabetes.feature_names)
y=diabetes.target

#2.Split into training and testing sets(80% train,20% test)
X_train,X_test,y_train,y_test=train_test_split(
X,y,test_size=0.2,random_state=42)

#3.Initialize and train the Decision Tree Regression.
regressor=DecisionTreeRegressor(max_depth=3, random_state=42)
regressor.fit(X_train,y_train)
y_pred=regressor.predict(X_test)
mse=mean_squared_error(y_test,y_pred)
r2=r2_score(y_test,y_pred)

print("----------Evaluation Metrics--------")
print(f"Mean Squared Error(MSE):{mse:.2f}")
print(f"R-squared Score(R2):    {r2:.2f}")
print("---------Learned Regression Rules(First few levels)-------")
tree_rules=export_text(regressor,feature_names=list(diabetes.feature_names))
print(tree_rules)
```

## Output:-

```
----------Evaluation Metrics--------
Mean Squared Error(MSE):3552.70
R-squared Score(R2):    0.33
---------Learned Regression Rules(First few levels)-------
|--- bmi <= 0.01
|   |--- s5 <= 0.01
|   |   |--- s5 <= -0.04
|   |   |   |--- value: [80.88]
|   |   |--- s5 >  -0.04
|   |   |   |--- value: [109.92]
|   |--- s5 >  0.01
|   |   |--- s4 <= 0.09
|   |   |   |--- value: [159.57]
|   |   |--- s4 >  0.09
|   |   |   |--- value: [256.33]
|--- bmi >  0.01
|   |--- bmi <= 0.07
|   |   |--- s6 <= 0.03
|   |   |   |--- value: [175.80]
|   |   |--- s6 >  0.03
|   |   |   |--- value: [230.52]
|   |--- bmi >  0.07
|   |   |--- s2 <= 0.02
|   |   |   |--- value: [291.22]
|   |   |--- s2 >  0.02
|   |   |   |--- value: [225.75]
```

**7.)Aim:-** Implement k-Nearest Neighbors (k-NN) classifier.

## Code:-

```
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
#1.Load the Iris dataset.
iris=load_iris()
X,y=iris.data, iris.target

#2.split into training and testing sets (80% train,20%test)
X_train,X_test,y_train,y_test=train_test_split(
X,y,test_size=0.2,random_state=42)

#3.Scale featuires
scaler = StandardScaler()
X_train_scaled=scaler.fit_transform(X_train)
X_test_scaled=scaler.transform(X_test)

#4.Initialize and train the k-NN Classifier(using 3 neighbors)
knn=KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train_scaled, y_train)

#5.Make predictions and Evaluate accuracy
y_pred=knn.predict(X_test_scaled)
accuracy=accuracy_score(y_test,y_pred)

print(f"KNN Classifier4 Accuracy (k=3):{accuracy*100:.2f}%")
```

## Output:-

```
KNN Classifier4 Accuracy (k=3):100.00%
```

**8.)Aim:-** Implement multiclass classification using logistic Regression or Decision Tree.

## Code:-

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics  import classification_report
from sklearn.preprocessing import StandardScaler

#1.Load the Iris dataset.
iris=load_iris()
X,y=iris.data, iris.target

#2.split into training and testing sets (80% train,20%test)
X_train,X_test,y_train,y_test=train_test_split(
X,y,test_size=0.2,random_state=42)

#3.Scale featuires
scaler = StandardScaler()
X_train_scaled=scaler.fit_transform(X_train)
X_test_scaled=scaler.transform(X_test)

#4.Train Multiclass Logistic Regression
model=LogisticRegression(random_state=42)
model.fit(X_train_scaled,y_train)

#5.Evaluate the Model.
y_pred=model.predict(X_test_scaled)

print("------------Multiclass Classification Report----------")
```

```
print(classification_report(y_test,y_pred,target_names=iris.target_names))
```

## Output:-

```
------------Multiclass Classification Report-------------------
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```